Time Control 50

This library require other libraries to be installed:

- Utils.sul

Chanelog

V2 [12.01.2026]

!!! Breaking changes

- depricated - ticker `TCO_INT_50` since it does not store enough data for most of the cases. Only `TCO_DINT_50` is used now. Because of that all functions work with `INT` are deleted and all function work with `DINT` renamed.

Description

This library provides timer that increments by 50ms or 10ms. It replaces Codesys `TIME()` function.

TCO - Short of Time Controls;

This a table on data types in this manual and to what types in GX Works2 it refer.

Type in manual	Type in GX Works 2
`INT`	Word[Signed]
`WORD`	Word[Unsigned]/Bit String[16-bit]
`DINT`	Double Word[Signed]
`DWORD`	Double Word[Unsigned]/Bit String[32-bit]
`BOOL`	Bit

TCO Ticker Setup

This library tris to implement it's own timer counter like in CoDeSys that is returned by function `TIME()`.

This library have two global variables that contain current timer (TICKER). Double Word[Unsigned]/Bit String[32-bit]

- `TCO_DINT_50` `DWORD`- Contain number of 50ms increments from PLC start in `DWORD` format stores approximately 3.4 years.
- `TCO_DINT_10` `DWORD`- Contain number of 10ms increments from PLC start in `DWORD` format stores approximately 1 year.

In order for this variable to start working we have to start TCO ticker, few things have to be done.

1. In main `POU_01` and all other programs you have to add at as a very first line:

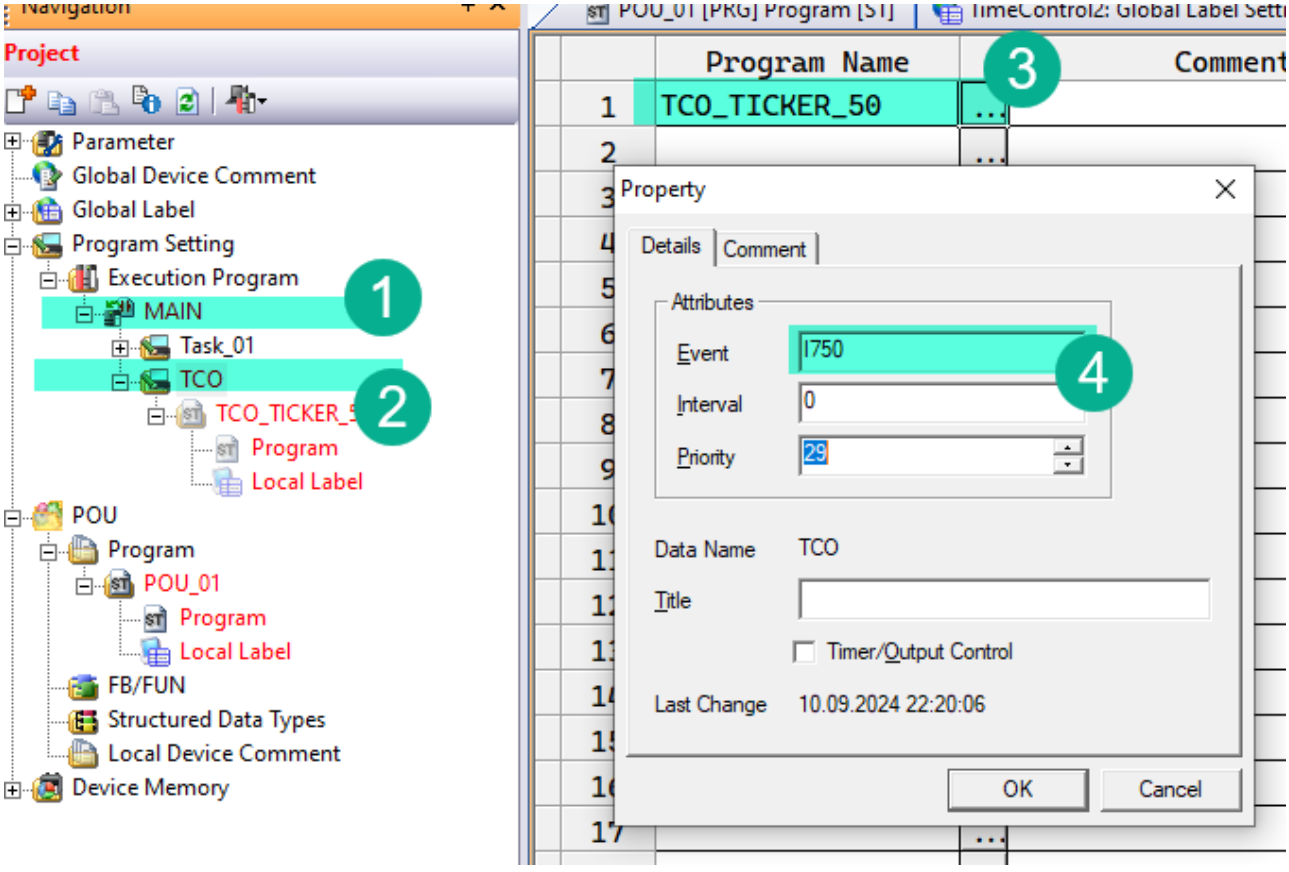
```
EI(TRUE);
```

This enables global interrupts that is used for TCO ticker.

2. In main program also add counters reset on PLC starts

```
EI(TRUE);
RST(M8002, TCO_DINT_50);
RST(M8002, TCO_DINT_10);
```

3. Right click in project tree *Program Settings/Execution Program/MAIN* (1), add new object type Task and name it TCO (2). With a right link on the new task created select properties and for event enter `I750` (4). This tells that this program will run every 50ms regardless main program execution time. Link for this task `TCO_TICKER_50` program (3) from TimeControl50 library.



If you want to setup ticker by 10ms use `I610` and select `TCO_TICKER_10` program. But it will probably make more load on a PLC.

TCO Conversion Functions

TCO_50_TO_SEC, TCO_50_TO_100MS, TCO_50_TO_MS, TCO_50_TO_MIN

These functions convert `TCO_DINT_50` or `TCO_INT_50` into seconds, minutes or 100ms increments.

Variable	Scope	Type	Description

Variable	Scope	Type	Description
TICKER	INPUT	DWORD	Time measured in 50ms intervals

All functions return INT and TCO_50_TO_MS returns DINT.

```
EI(TRUE);
iCurrentSeconds := TCO_50_TO_SEC(3520); (* 176 sec *)
```

Where iCurrentSeconds is INT and diCurrentSeconds is DWORD.

MIN_TO_TCO_50, SEC_TO_TCO_50

Convert minutes or seconds to 50ms intervals.

MIN_TO_TCO_50

Variable	Scope	Type	Description
MINUTES	INPUT	INT	Time measured in 50ms intervals

SEC_TO_TCO_50

Variable	Scope	Type	Description
SECONDS	INPUT	INT	Time measured in 50ms intervals

Both functions return DWORD.

```
EI(TRUE);
diTCO50 := MIN_TO_TCO_50(1); (* 1200 *)
```

TCO_50_DIFF

This function returns a difference in 50ms increments between current time and saved point.

```
EI(TRUE);

IF MEP(M0) THEN
    (* iStart is DWORD *)
    iStart := TCO_DINT_50;
END_IF;

IF MEF(M0) THEN
    (* iEnd is INT *)
    iEnd := TCO_50_TO_SEC(TCO_50_DIFF(iStart, TCO_DINT_50));
END_IF;
```

This program example saves in `iEnd` how many seconds `M0` was in `TRUE` state, since `MEP` is a raise trigger and `MEF` is a fall trigger.

General Functions And Blocks

TCO_50_BLINK

Is a classical IEC 61131-3 block. It starts with `TIMELOW` interval. It also unlike CoDeSys BLINK turn output off in `IN` is `false`

TCO_50_BLINK

Variable	Scope	Type	Description
`TIMELOW`	INPUT	DWORD	Time for output `Q` to be OFF in 50ms interval
`TIMEHIGH`	INPUT	DWORD	Time for output `Q` to be ON in 50ms interval
`IN`	INPUT	Bit	Enabled this timer to start working
`Q`	OUTPUT	Bit	Current state

```
VAR
    fbBlink: TCO_50_BLINK;
END_VAR

fbBlink(TIMELOW := MIN_TO_TCO_50(1440), TIMEHIGH := MIN_TO_TCO_50(1440), EN :=
X0);

Y0 := fbBlink.Q;     (* One day motor one *)
Y1 := NOT fbBlink.Q; (* One day motor two *)
```

This example rotates motors by 24 hours intervals